

Relational Databases

4. Basics of transaction management

Content

- 4.1. Transaction Management and Concurrency Models
- 4.2. Lock Types
- 4.3. Blocking in the System
- 4.4. Deadlocks
- 4.5. Transactions and Triggers

4.1. Transaction Management and Concurrency Models

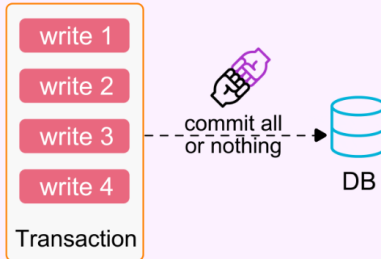
Need for transactions

- Majority of databases are **multi-user**
 - **Concurrent access to the same data may induce different types of anomalies**
 - Errors may occur in the DBMS or its environment
- The **Database Engine should handle simultaneous workloads from multiple users** and provide the required level of data consistency when users query and modify the same data.
- DBMS must support **ACID** properties:
Atomicity, Consistency, Isolation, Durability

ACID

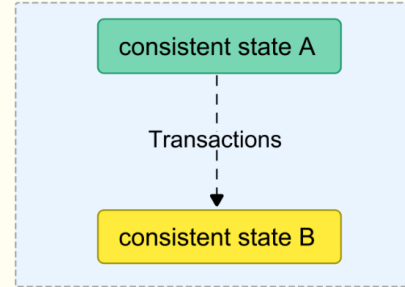
Atomicity

All or nothing



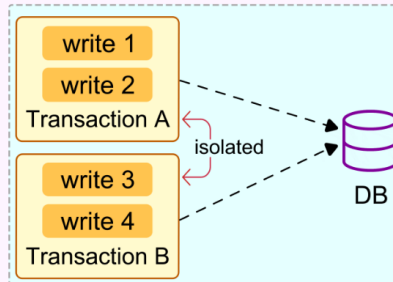
Consistency

Preserving database invariants



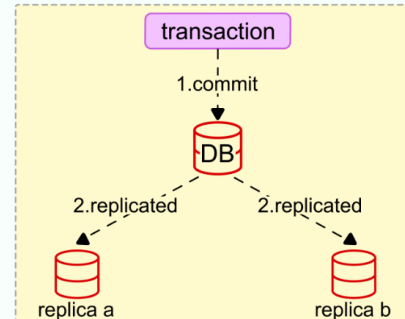
Isolation

Concurrent transactions are isolated from each other



Durability

Data is persisted after transaction is committed even in a system failure



Acid: Atomicity

- *Atomicity* guarantees that each transaction executes as a single unit of work using an “**all or nothing**” approach.
- All changes done within a transaction are either committed or rolled back in full.
- Consider the classic example of transferring money between checking and savings bank accounts. That action consists of two separate operations:
 - decreasing the balance of the checking account
 - increasing the balance of the savings account.
- Transaction atomicity guarantees that **both operations either succeed or fail together**, and a system will never be in an inconsistent state where money was deducted from the checking account but never added to the savings account.

aCid: Consistency

- *Consistency* ensures that
 - **any database transaction brings the database from one consistent state to another**
 - none of the defined database rules and constraints (for instance foreign key constraints) were violated.

acid: Isolation

- *Isolation*
 - ensures that the **changes made in the transaction are isolated and invisible to other transactions until the transaction is committed.**
 - guarantee that the concurrent execution of multiple transactions brings the system to the same state as if those transactions were executed serially.
 - However, **in most database systems, such a requirement is often relaxed and controlled by *transaction isolation levels***, which we will discuss later in this chapter.

aciD: Durability

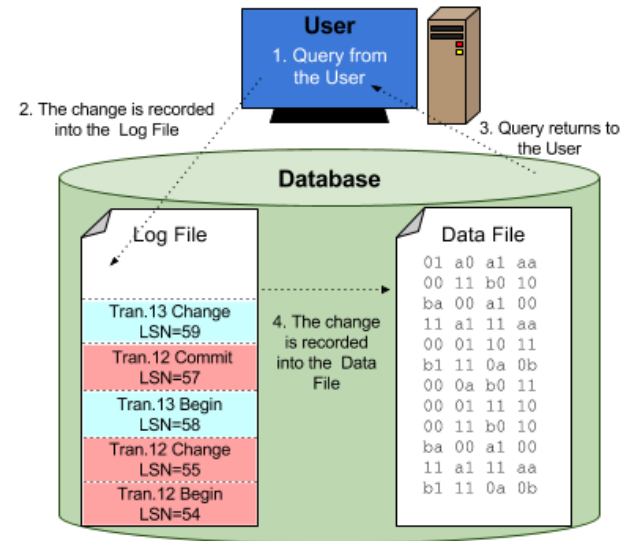
- Guarantees that **after a transaction is committed, all changes done by the transaction stay permanent and will survive a system crash.**
- SQL Server achieves durability by using **write-ahead logging to harden log records in the transaction log** (see next slides).
- A transaction is not considered to be committed until all log records generated by the transaction are hardened in the log file.

Transaction Logfile

- Logfile registers
 - a unique log sequence number
 - a unique transaction identifier
 - a marking to denote the start of a transaction, along with the transaction's start time and indication whether the transaction is read only or read/write
 - identifiers of the database records involved in the transaction, as well as the operation(s) they were subjected to
 - **before images** of all records that participated in the transaction
 - **after images** of all records that were changed by the transaction
 - the current state of the transaction (active, committed or aborted)

Transaction Logfile

- Logfile may also contain checkpoints
 - moments when buffered updates by active transactions, as present in the database buffer, are written to disk at once
- Write ahead log strategy
 - all updates are registered on the logfile before written to disk
 - before images are always recorded on the logfile prior to the actual values being overwritten in the physical database files



Transaction Logfile

Tid	Time	Operation	Object	Before image	After image	pPtr	nPtr
T1	10:12	START				0	2
T1	10:13	UPDATE	STAFF SL21	(old value)	(new value)	1	8
T2	10:14	START				0	4
T2	10:16	INSERT	STAFF SG37		(new value)	3	5
T2	10:17	DELETE	STAFF SA9	(old value)		4	6
T2	10:17	UPDATE	PROPERTY PG16	(old value)	(new value)	5	9
T3	10:18	START				0	11
T1	10:18	COMMIT				2	0
	10:19	CHECKPOINT	T2, T3				
T2	10:19	COMMIT				6	0
T3	10:20	INSERT	PROPERTY PG4		(new value)	7	12
T3	10:21	COMMIT				11	0

Transaction Logfile

Use Northwind

-- Employee2 was used in Chapter 5: Indexes.

```
UPDATE Employee2 SET salary = 5000 WHERE EmployeeID = 1
```

```
DELETE FROM Employee2 WHERE EmployeeID = 50
```

-- If you want to see the contents of a database's transaction log,

-- i.e., what transactions have occurred – you can query the log

-- This is an undocumented but very useful function that reads the active transaction log.

```
SELECT [Transaction ID], [Transaction Name], [Operation], [Context], [AllocUnitName],  
       [Transaction SID], [Begin Time],[End Time], [Transaction Name], [Description]  
FROM fn_dblog(NULL, NULL);
```

-- You can also filter by operation type

-- UPDATE

```
SELECT * FROM fn_dblog(NULL, NULL) WHERE [Operation] LIKE 'LOP_MODIFY_ROW%';
```

-- INSERT

```
SELECT * FROM fn_dblog(NULL, NULL) WHERE [Operation] = 'LOP_INSERT_ROWS';
```

-- DELETE

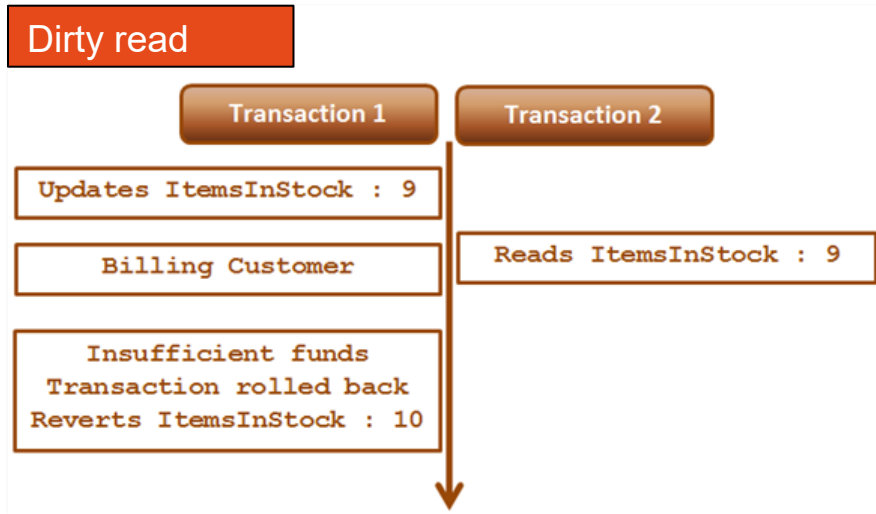
```
SELECT * FROM fn_dblog(NULL, NULL) WHERE [Operation] = 'LOP_DELETE_ROWS';
```

Problems with isolation (1)

- The **isolation requirements are the most complex to implement** in multi-user environments.
- Even though it is possible to completely isolate different transactions from each other, this **could lead to a high level of blocking and other concurrency issues** in systems with volatile data.
- SQL Server addresses this situation by introducing several **transaction isolation levels** that **relax isolation requirements at the cost of possible concurrency phenomena** related to read data consistency:

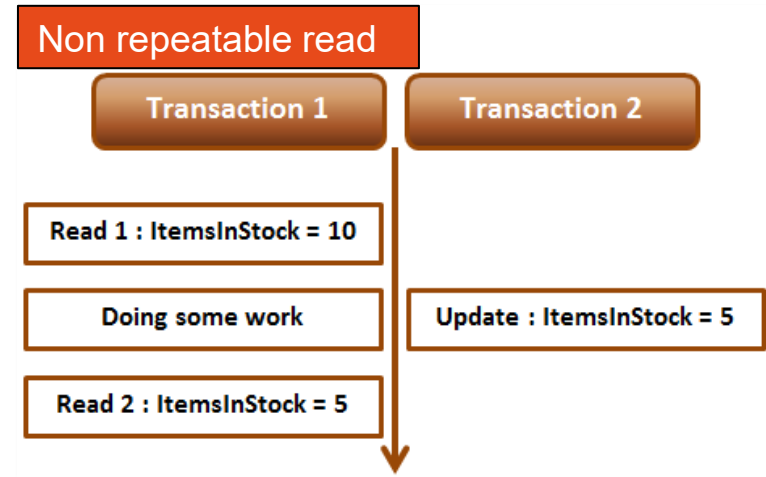
Problems with isolation (2)

- **Dirty Reads:** A transaction reads uncommitted (dirty) data from other uncommitted transactions.



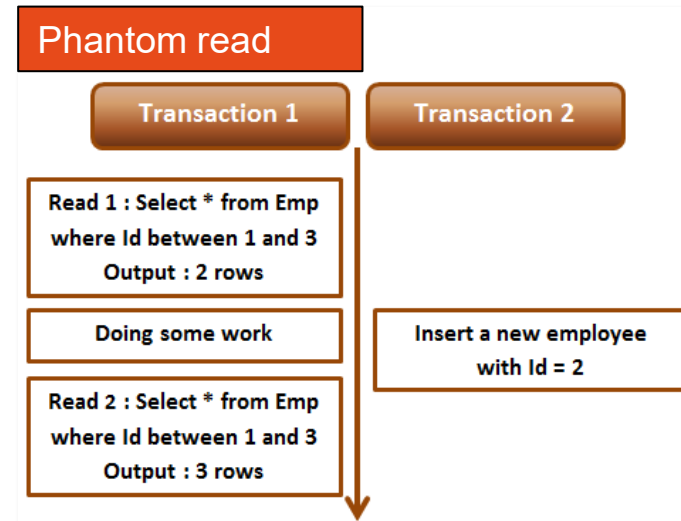
Problems with isolation (2)

- **Non-Repeatable Reads:** Subsequent attempts to read the same data from within the same transaction return different results. This data inconsistency issue arises when the other transactions modified, or even deleted, data between the reads done by the affected transaction.



Problems with isolation (3)

- **Phantom Reads:** This phenomenon occurs when subsequent reads within the same transaction return new rows (the ones that the transaction did not read before).
- This happens when another transaction inserted new data in between the reads done by the affected transaction.



Problems with isolation (3)

→ Solutions to all isolation problems:
pessimistic and optimistic concurrency



Pessimistic Concurrency

- Assumes multiple users want to modify the same data
- Database Engine prevents overwriting by locking data upon first access/update
- Other sessions are blocked until the first transaction completes

Optimistic Concurrency

- Assumes simultaneous updates are rare
- Data is not locked, but **write–write conflicts** may occur
- Conflicting transactions are rolled back

Example Scenario: Money Transfer and Withdrawal

- Pessimistic: first session locks checking account
→ second session waits
- Optimistic: no blocking, but one transaction fails with conflict error → error has to be caught by developer and transaction has to be restarted.

Comparison of Concurrency Models

- Pessimistic: prevents conflicts but may introduce blocking
- Optimistic: avoids blocking but requires conflict handling and overhead
- SQL Server supports both via transaction isolation levels

Comparison: Pessimistic vs Optimistic Concurrency

Aspect	Pessimistic Concurrency	Optimistic Concurrency
Assumption	Conflicts are likely → lock data	Conflicts are rare → no locks
Behavior	Blocks other sessions until transaction completes	Allows concurrent access, detects conflicts at commit
Pros	Prevents data conflicts, ensures consistency	Higher concurrency, no blocking
Cons	May cause blocking and reduced throughput	Requires conflict detection and retry, more overhead

Transaction Isolation Levels in SQL Server

- SQL Server supports **six different transaction** isolation levels with disk-based tables.
- **Isolation levels relax the strict isolation requirements at the cost of possible data consistency issues** related to read operations.
- These levels control the concurrency models used by transactions.

Concurrency Models in Isolation Levels

- Pessimistic Isolation Levels
 - READ UNCOMMITTED
 - READ COMMITTED
 - REPEATABLE READ
 - SERIALIZABLE
- Rely strictly on locking to prevent other sessions from accessing or modifying data.

Pessimistic Isolation Levels: reader locks

- **Read uncommitted:** reader doesn't ask for shared lock (= a lock that can be shared by >1 transactions, see further)
- **Read committed:** reader keeps shared lock on the row until it is obtained, not until end of statement (in case of multiple row result set) or end of transaction (= short-term read lock)
- **Repeatable read:** reader claims shared lock and holds it until end of transaction (= long-term read lock)
- **Serializable:** reader locks all keys (current and future) that correspond to WHERE-clause

Optimistic Isolation Level SNAPSHOT

- Uses row versioning and copies old versions of modified rows to a special area in tempdb called the version store.
- Other sessions read the old, committed versions of the rows from the version store rather than being blocked.
- SQL Server still acquires locks on updated rows to prevent other sessions from updating the same rows simultaneously.

The READ COMMITTED SNAPSHOT Level

- This 6th isolation level combines both optimistic and pessimistic concurrency models.
- Readers (SELECT queries) use row versioning (optimistic).
- Writers (INSERT, UPDATE, DELETE queries) rely on locking (pessimistic).
- Note on Implementation: It is technically a database option (READ_COMMITTED_SNAPSHOT) that changes the default behavior of readers in the READ COMMITTED isolation level.
- Default State: This option is enabled by default in Microsoft Azure SQL Databases and disabled by default in regular versions of SQL Server

SNAPSHOT vs. READ COMMITTED SNAPSHOT

Feature	READ COMMITTED SNAPSHOT (RCSI)	SNAPSHOT
Enabled by	READ_COMMITTED_SNAPSHOT ON	ALLOW_SNAPSHOT_ISOLATIO N ON
Used automatically	Yes (for READ COMMITTED)	No, must set isolation level
Snapshot scope	Per statement	Per transaction
Repeatable reads	No	Yes
Prevents reader/writer blocking	Yes	Yes
Update conflict detection	No	Yes
Consistent view throughout transaction	No	Yes

Setting the Isolation Level on session level

- You can set the transaction level on the session level using the `SET TRANSACTION ISOLATION LEVEL` statement.
- Most client libraries use `READ COMMITTED` as the default level (or `READ COMMITTED SNAPSHOT` when the option is enabled).

```
set transaction isolation level repeatable read
begin transaction
select orderid, productid, quantity, unitprice
from orderdetails
commit
```

Setting the Isolation Level on table level

The isolation level can also be set on table level using **table hints**:

```
select c.companyname, sum(od.quantity*od.unitprice)
from customers c with (readcommitted)
join orders o with (serializable) on c.customerid = o.customerid
join orderdetails od on o.orderid = od.orderid
group by c.companyname
```

The famous NOLOCK level is a synonym for READ UNCOMMITTED and is very useful in a production environment if you want to execute a long-running SELECT statement and want to avoid writers (insert/update/delete) have to wait until finishing:

```
select orderid, productid, quantity, unitprice
from orderdetails with (NOLOCK)
```

Transaction Types in SQL Server

- SQL Server recognizes three main types of transactions:
 - **Explicit**
 - **Autocommitted** (also called AutoCommit)
 - **Implicit**

1. Explicit Transactions

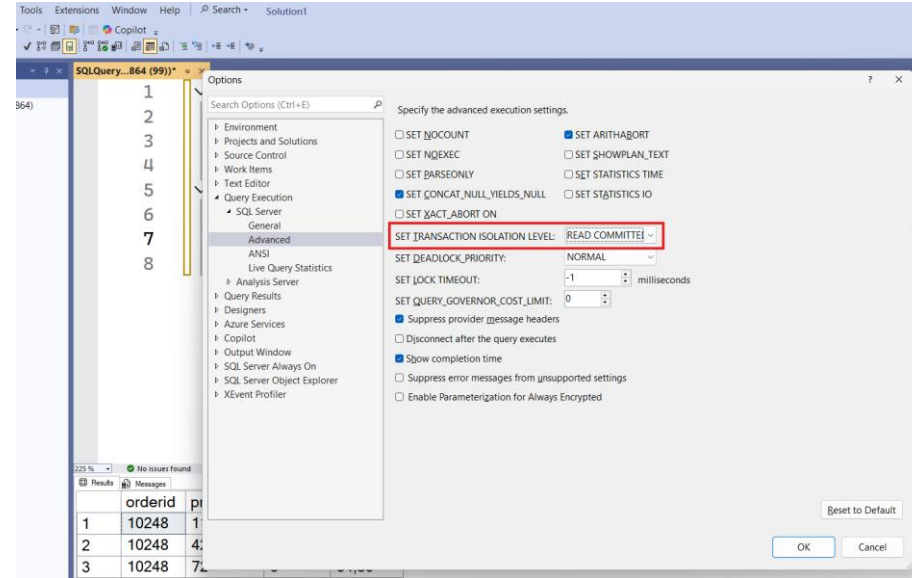
- Transactions explicitly controlled by the code.
- Start: Initiated using the **BEGIN TRAN** statement.
- Duration: Remain active until explicitly terminated by either **COMMIT** (to save changes) or **ROLLBACK** (to undo changes).
- Efficiency: More efficient for logic involving multiple data modification statements compared to Autocommitted transactions.
- Leads to more efficient logging because log writes are batched.

2. Autocommitted Transactions (The Default)

- The transaction type SQL Server uses **when no explicit transaction is active.**
- Behavior: SQL Server **automatically starts a transaction before a statement executes and commits it immediately after** the statement completes.
- Scope: Works on a per-statement level, NOT a per-module (e.g., stored procedure) level.
 - Example: A stored procedure with five statements executes five separate autocommitted transactions.
- Failure Consequence: **If a stored procedure fails mid-execution, previously autocommitted transactions are NOT rolled back, which can lead to logical data inconsistencies.**
- Performance Hit:
 - **Less efficient due to excessive logging overhead.**
 - Every single statement generates extra log records for the implicit BEGIN TRAN and COMMIT operations.
 - Each implicit COMMIT forces SQL Server to flush the log buffers (caching up to 60 KB of log records) to disk, introducing a synchronous I/O operation (log buffer flush) which degrades performance.

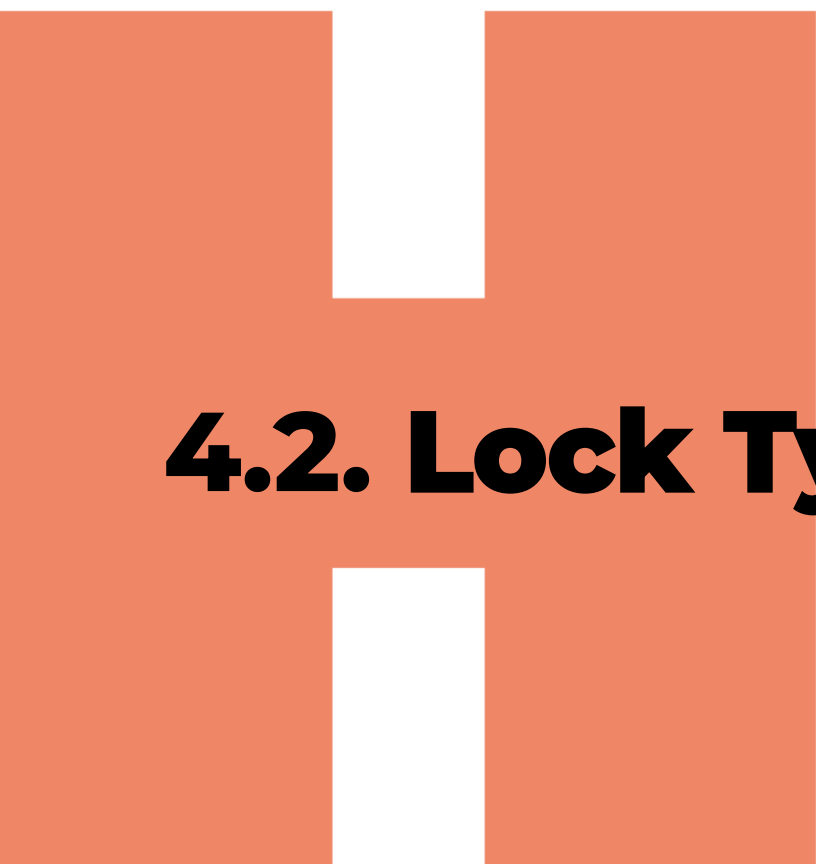
3. Implicit transactions

an **optional setting** on the session that makes certain statements (like DML) automatically start an explicit transaction, which must still be explicitly committed or rolled back.



Error handling

- Error handling in SQL Server is a tricky subject, especially with transactions involved.
- See script `4.01.Error Handling.sql` with autocommit (= default) for both tests. Make sure you understand what happens.
- **Exercise:** use explicit transactions for both tests (all 3 deletes in a single transaction). Make sure that if one delete fails all deletes are rolled back. Which approach (test 1 or test 2) do you prefer?



4.2. Lock Types

Major Lock Types

- SQL Server uses locking to support isolation requirements
- Locks are acquired and held on *resources*, such as data rows, pages, tables, databases, ...
- By default, SQL Server uses row-level locking
- Key attribute in the lock structure is the *lock type*.
- Most important locks for the developer to consider are: Exclusive locks and Shared locks.

Exclusive (X) Locks

- Are acquired by writers—INSERT, UPDATE, DELETE, and MERGE
- Those queries acquire exclusive (X) locks on the affected rows and hold them until the end of the transaction.
- Only one session can hold an exclusive (X) lock on the resource at any given point in time.
- This behavior enforces the most important concurrency rule in the system—multiple sessions cannot modify the same data simultaneously.
- Transaction isolation levels do not affect exclusive (X) lock behavior.

Shared (S) locks

- are acquired by the readers—SELECT queries
- are compatible with each other
- Multiple sessions can hold shared (S) locks on the same resource.

Lock Compatibility Matrix

Type of lock(s) currently held on object

*Type of lock
requested*

	<i>unlocked</i>	<i>shared</i>	<i>exclusive</i>
<i>unlock</i>	-	yes	yes
<i>shared</i>	yes	yes	no
<i>exclusive</i>	yes	no	no

Most important lock compatibility rules

Exclusive (X) locks

- are incompatible with each other and any other lock types.
- Multiple sessions cannot update the same row simultaneously.
- Readers that acquire shared (S) locks cannot read uncommitted rows with exclusive (X) locks held.

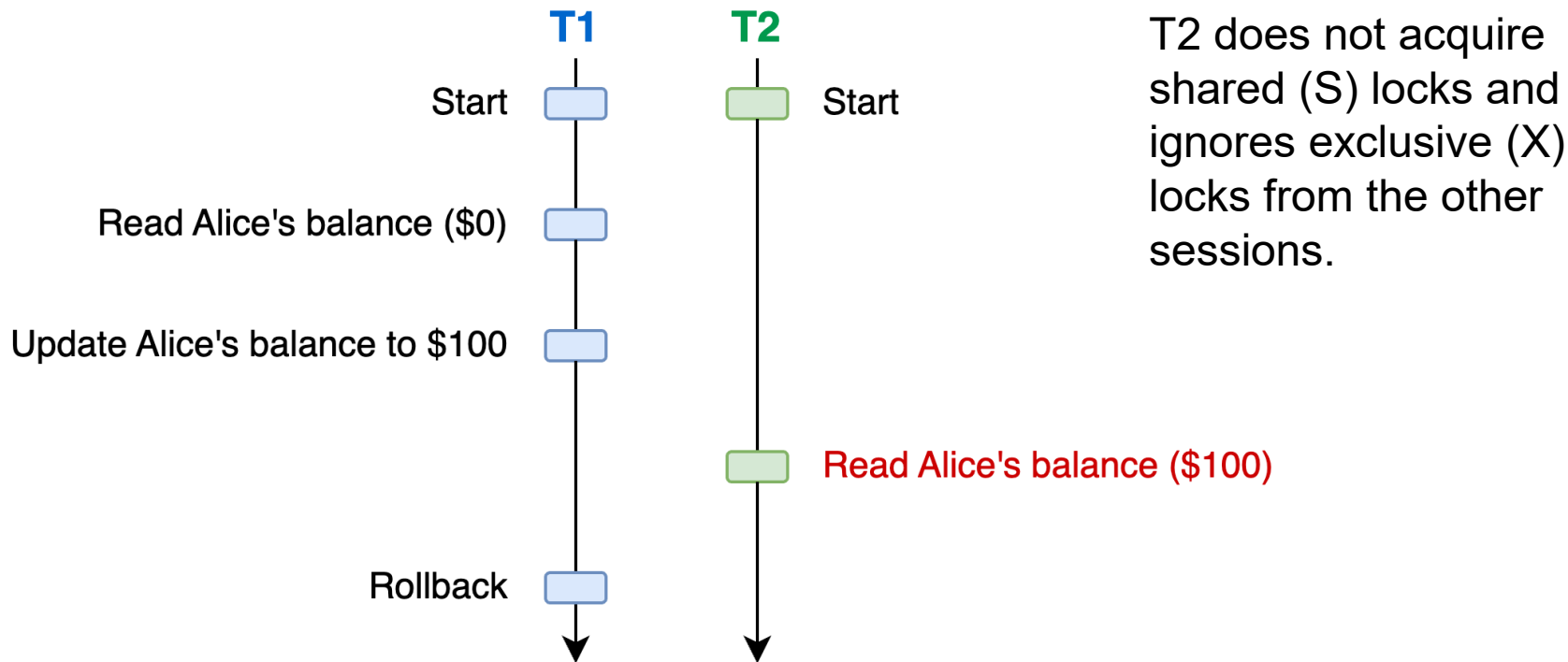
Transaction Isolation Levels and Data Consistency

Dirty Reads:

- arise **when transaction reads uncommitted (dirty) data from other uncommitted transactions.**
- It is unknown if those active transactions will be committed or rolled back or if the data is logically consistent.
- From the locking perspective, this phenomenon could occur in the **READ UNCOMMITTED** isolation level when sessions do not acquire shared (S) locks and ignore exclusive (X) locks from the other sessions.
- All other isolation levels are immune from dirty reads:
 - **Pessimistic** isolation levels use shared (S) locks and **are blocked when trying to access uncommitted rows** with exclusive (X) locks held on them.
 - **Optimistic** isolation levels **read old** (previously) **committed versions** of the rows from the version store.

Transaction Isolation Levels and Data Consistency

Dirty Reads - Example



Transaction Isolation Levels and Data Consistency

Non-Repeatable Reads:

- **Subsequent attempts to read the same data from within the same transaction return different results.**
- **Arises when the other transactions modified or even deleted data between reads.**
- Consider a situation where you render a report that displays a list of the orders for a specific customer along with some aggregated information (for example, total amount spent by customer on a monthly basis). If another session modifies or perhaps deletes the orders in between those queries, the result sets will be inconsistent.

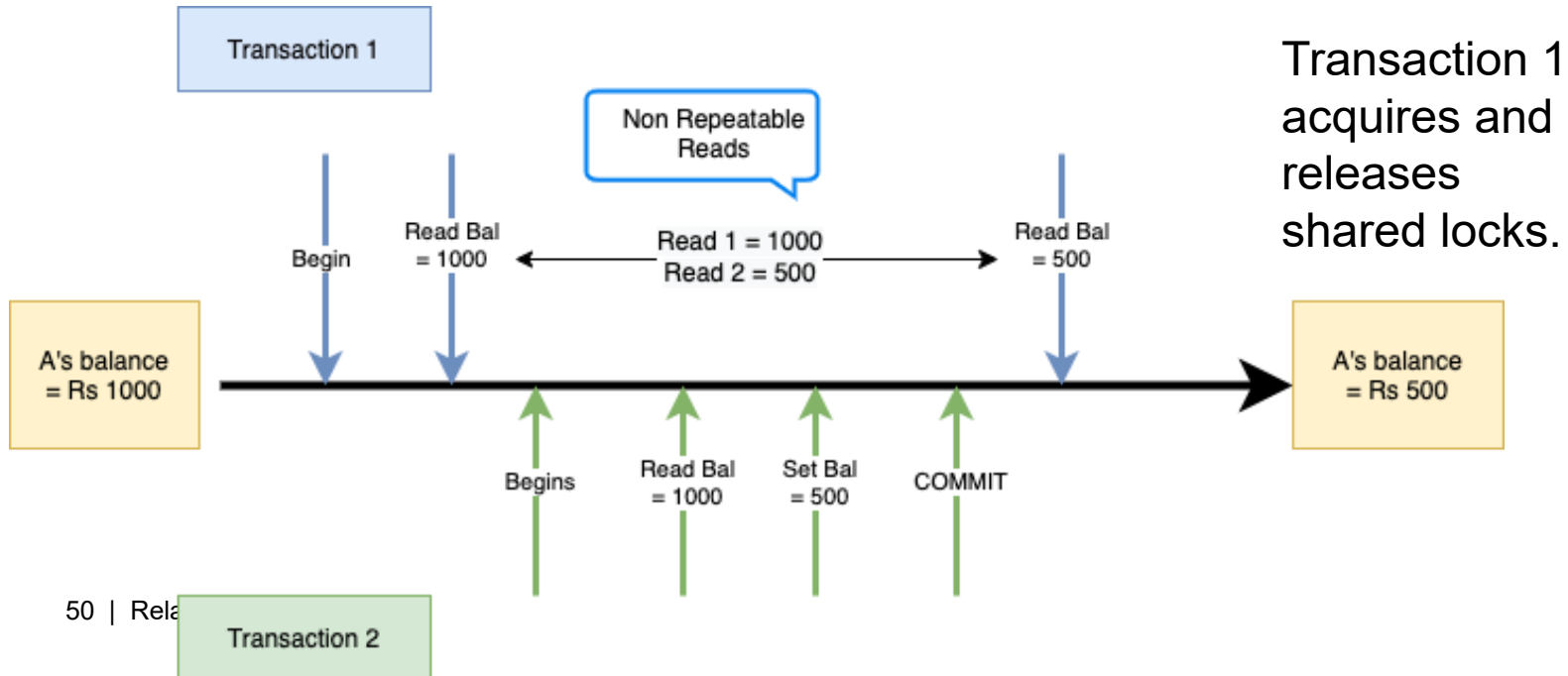
Transaction Isolation Levels and Data Consistency

Non-Repeatable Reads (cont'd):

- From the locking standpoint this could occur **when sessions don't protect/lock the data in between reads.**
- This could happen
 - in the READ UNCOMMITTED and READ COMMITTED SNAPSHOT isolation levels, which do not use shared (S) locks,
 - in the READ COMMITTED isolation level when sessions acquire and release shared (S) locks immediately.
- REPEATABLE READ and SERIALIZABLE isolation levels hold the shared (S) locks until the end of the transaction, which prevents data modifications once data is read.
- The SNAPSHOT isolation level is also immune from this phenomenon as it works with a snapshot of the data at the time when the transaction started.

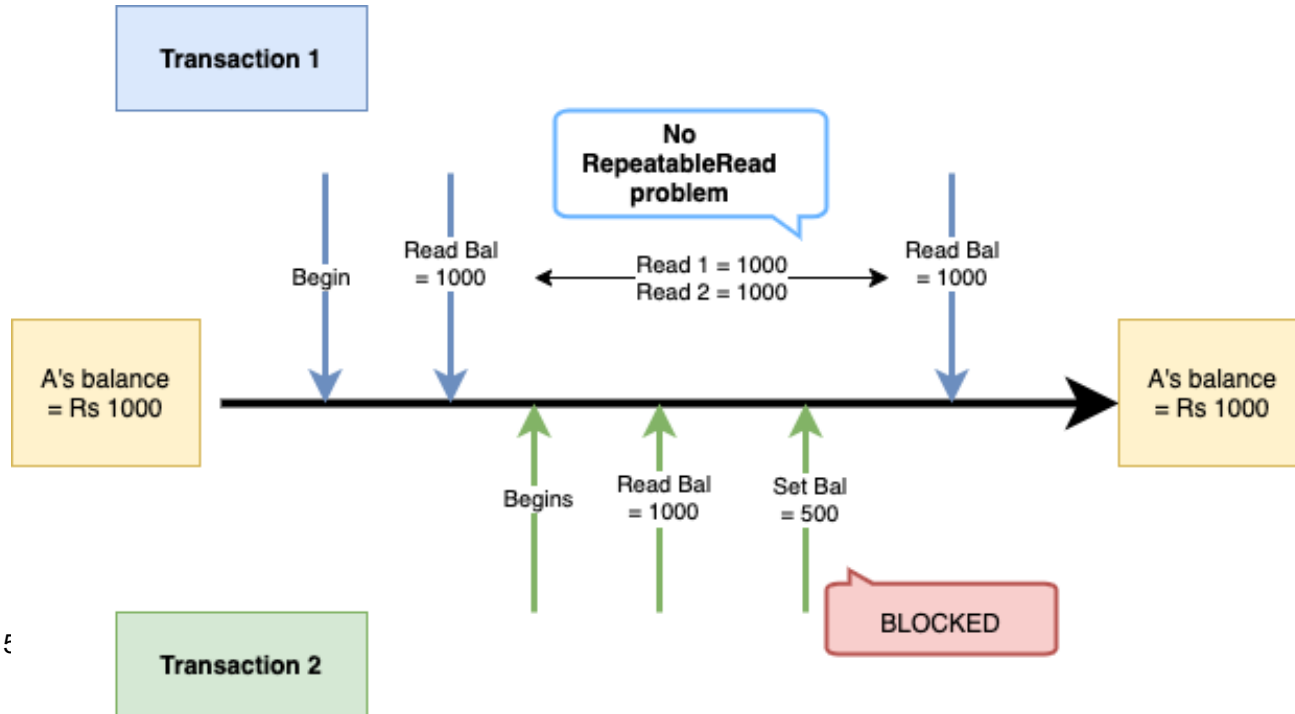
Transaction Isolation Levels and Data Consistency

Non-Repeatable Reads – Example – Part 1 :



Transaction Isolation Levels and Data Consistency

Non-Repeatable Reads – Example – Part 2 :



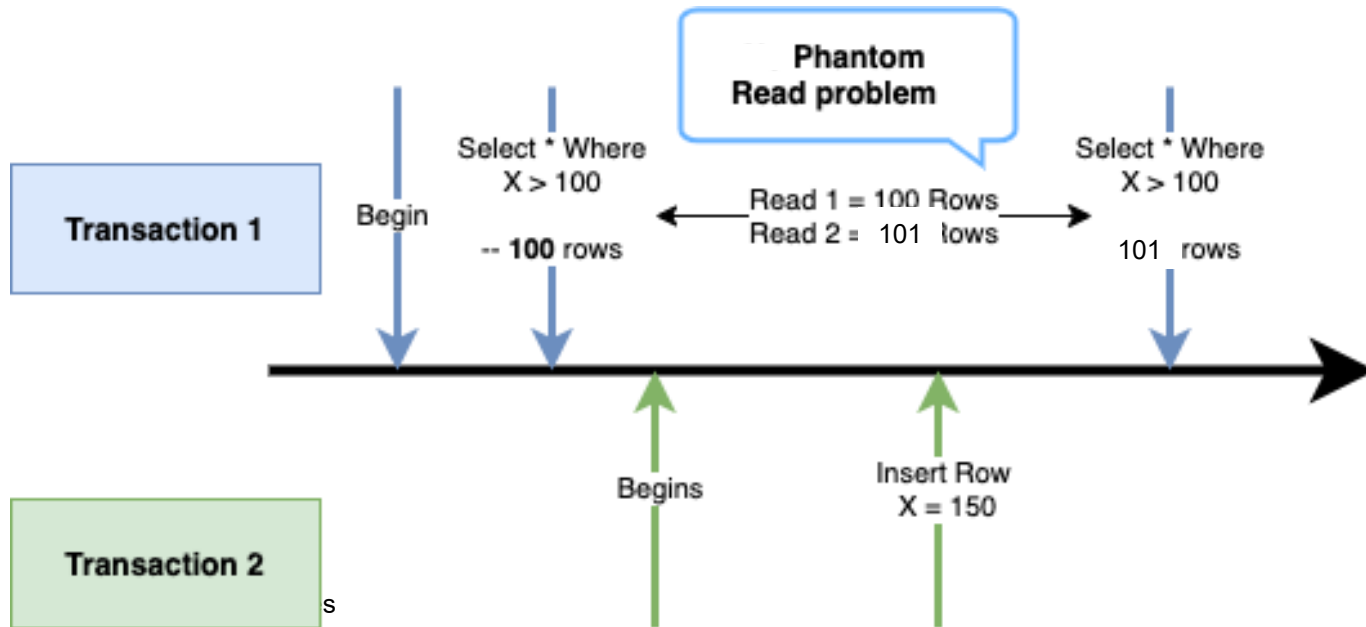
Transaction Isolation Levels and Data Consistency

Phantom Reads:

- occurs **when subsequent reads within the same transaction return new rows** (ones that the transaction did not read before).
- Think about an example when another session inserted a new order in between two queries of `count(*)` on the orders table.
- Only SERIALIZABLE and SNAPSHOT avoid Phantom Reads:
 - SERIALIZABLE uses range locks
 - SNAPSHOT accesses a snapshot of the data at the time when the transaction starts.

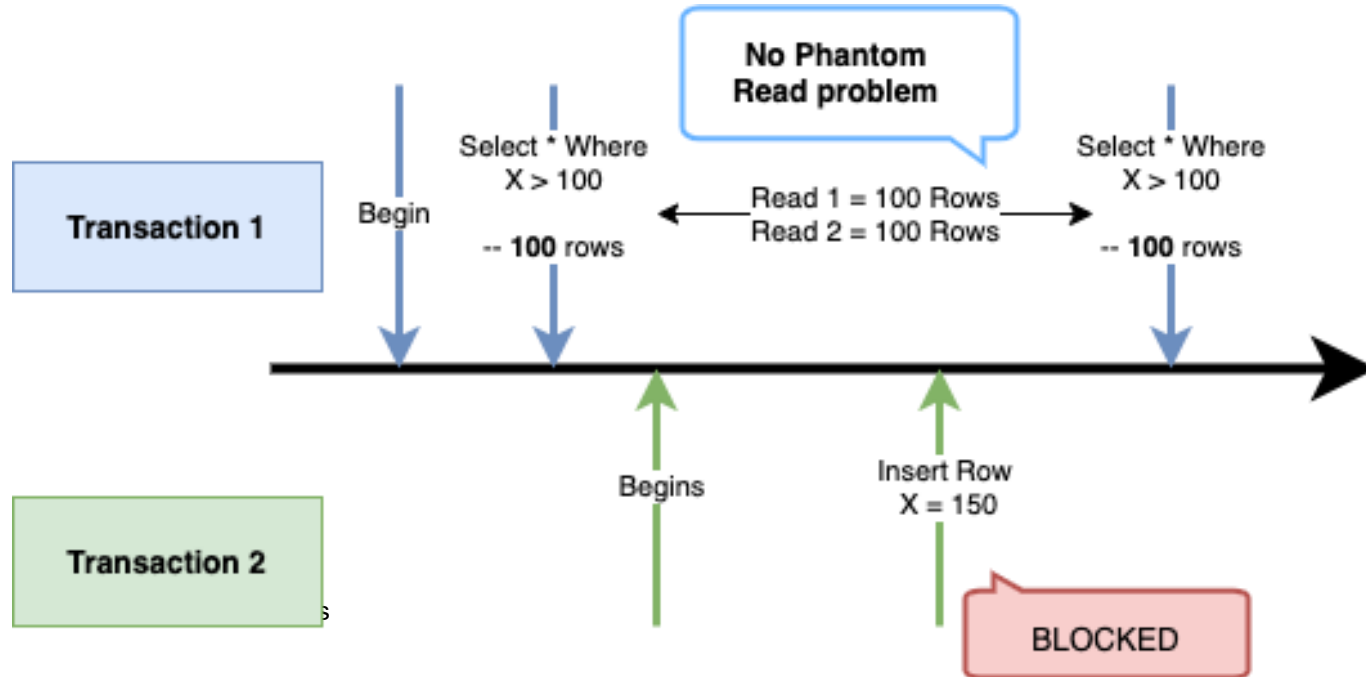
Transaction Isolation Levels and Data Consistency

Phantom Reads – Example – Part 1:



Transaction Isolation Levels and Data Consistency

Phantom Reads – Example – Part 2:



Transaction Isolation Levels and Data Consistency

Duplicated Reads:

- occurs when a query returns the same row multiple times.
- Think about a query that returns a list of orders for a specific time interval, scanning the index on the OrderDate column during execution.
- If another query changes the OrderDate value, moving the row from the processed (scanned) to non-processed part of the index, such a row will be read twice.
- This condition is similar to non-repeatable reads and can occur when readers do not hold shared (S) locks after rows are read in READ UNCOMMITTED and READ COMMITTED isolation levels (row-level locking!).

Transaction Isolation Levels and Data Consistency

Skipped Rows:

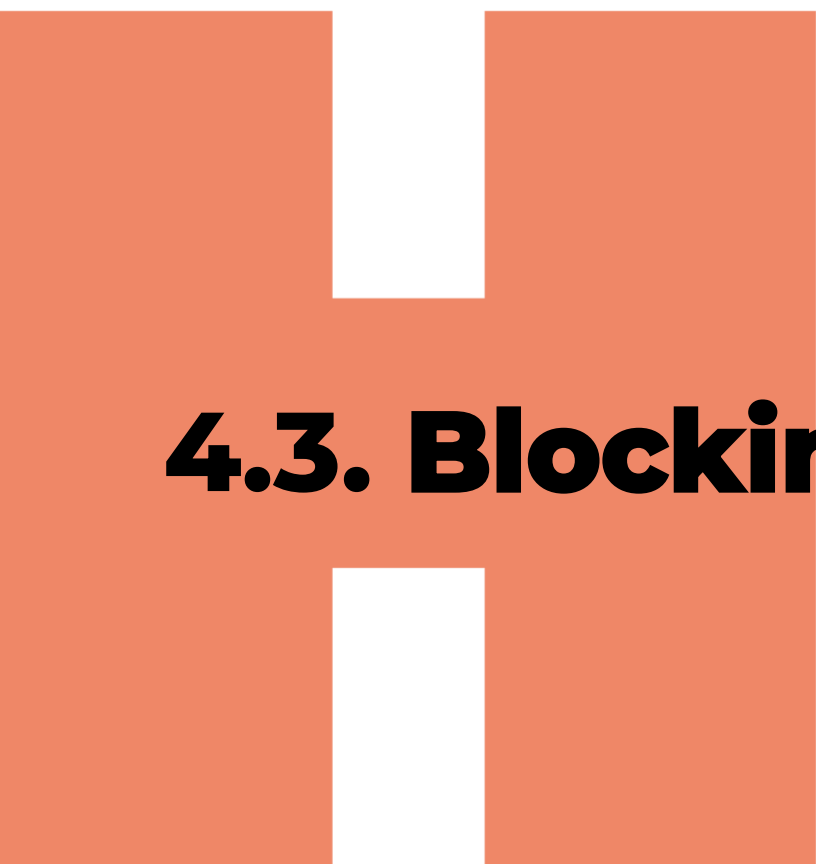
- occurs when queries do not return some of the rows.
- It could occur in a similar condition with duplicated reads as just described if rows have been moved from the non-processed to the processed part of the index.
- The SERIALIZABLE isolation level, which locks the index-key range interval, and optimistic isolation levels—READ COMMITTED SNAPSHOT and SNAPSHOT—avoid skipped rows.

Transaction Isolation Levels and Data Inconsistency Anomalies

	Dirty Reads	Non-Repeatable Reads	Duplicated Reads	Phantom Reads	Skipped Rows
READ UNCOMMITTED	Yes	Yes	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes	Yes	Yes
REPEATABLE READ	No	No	No	Yes	Yes
SERIALIZABLE	No	No	No	No	No
READ COMMITTED SNAPSHOT	No	Yes	No	Yes	No
SNAPSHOT	No	No	No	No	No

Exercise

- Which concurrency problem is illustrated in script 4.02 session1.sql and 4.02 session2.sql. Pay attention to the “interwoven” order of execution as specified in the comments.
- Explain why we get this problem.
- How can you solve this problem?



4.3. Blocking in the System

Blocking: The Common Concurrency Problem

- **Definition:** A concurrency issue where multiple active queries or sessions block each other from accessing or modifying a shared resource.
- **Negative Impact:**
 - Increases query execution time.
 - Introduces timeouts for users and applications.
 - Significantly degrades the overall user experience.

Causes and Nature of Blocking

- **Mechanism:** Blocking occurs when **multiple sessions compete for the same resource**, and one session holds a lock that another session requires.
- **Expected vs. Unexpected Blocking:**
 - **Expected & Normal:** It is a correct and necessary behavior in some cases (e.g., preventing two sessions from simultaneously updating the exact same row).
 - **Unexpected & Excessive:** Occurs when queries acquire locks that are **unnecessary** or held for too long.
- **The Problem: Excessive blocking** is the issue, often manifesting to the end user as:
 - **General system slowness.**
 - **Query timeouts.**
 - **Deadlocks** (a specific, severe type of blocking, see further).

Troubleshooting Approach

- **Initial Assessment:** System slowness is not always a sign of blocking (many factors can impact performance), but blocking is a common contributor.
- **Holistic View:** During the initial performance troubleshooting phase, you should take a holistic view of the system to identify the most critical issues.
- **Focus of this Topic:** The general troubleshooting approach for blocking is key:
 - **Analyze blocking conditions in real time.**
 - Collect information for further analysis.

Troubleshooting Blocking Issues

The 3-Step Troubleshooting Approach

1. **Detect:** Identify the specific queries and sessions that are involved in the blocking chain.
2. **Find the Cause:** Determine the **root reason** why the blocking is occurring (e.g., long-running transaction, missing index, or unnecessary locks).
3. **Fix:** Implement the necessary changes to resolve the root cause of the issue.

Example: real-time blocking trouble shooting

- Code that leads to blocking condition (both sessions access same record!).
- See scripts
 - 4.03 session 1.sql
 - 4.03 session 2.sql
 - 4.03 monitor.sql
- Run the scripts in 3 parallel sessions and execute step by step as follows:

```

6.03 sessi...r864 (53))
-- SESSION 1 (blocking session)
-- step 1
select @@spid -- session id
go
use Northwind
go
begin tran
delete from customers
where customerid = 'PARIS'

-- step 4
rollback

```

Results	Messages
(No column name)	
1	53

```

6.03 sessi...ecuting...
-- SESSION 2 (blocked session)
-- step 2
select @@spid -- session id
go
use Northwind
go
select customerid, companyname
from customers
where customerid = 'PARIS'

```

Results	Messages
(No column name)	
1	81

```

6.03 moni...r864 (52))
-- MONITOR
-- step 3
-- All active requests to Lock Manager:
select resource_type,request_mode,request_type,
request_status,request_session_id
from sys.dm_tran_locks;
-- last line shows 2nd session
-- is waiting for a shared lock

-- show session id that blocks session 2
select session_id, wait_duration_ms,blocking_session_id
from sys.dm_os_waiting_tasks where session_id > 50
and blocking_session_id is not null

-- show more information
select db name(TL1.resource database id) as [DB Name]

```

Results

Messages

	resource_type	request_mode	request_type	request_status	request_session_id
1	DATABASE	S	LOCK	GRANT	81
2	DATABASE	S	LOCK	GRANT	53
3	DATABASE	S	LOCK	GRANT	52
4	PAGE	IS	LOCK	GRANT	81
5	PAGE	IX	LOCK	GRANT	53
6	OBJECT	IS	LOCK	GRANT	81
7	OBJECT	IX	LOCK	GRANT	53
8	KEY	X	LOCK	GRANT	53
9	KEY	S	LOCK	WAIT	81

	session_id	wait_duration_ms	blocking_session_id
1	81	6018	53

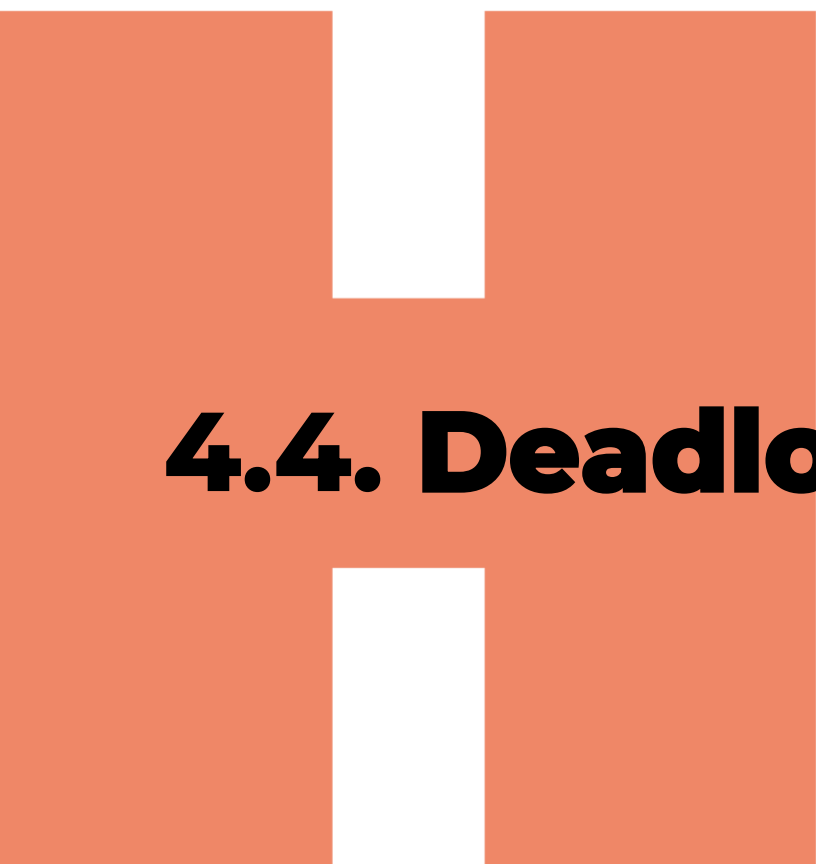
	DB Name	Session	Mode	Status	Wait (ms)	sql
1	Northwind	81	S	WAIT	6083	SELECT [customerid],[companyname] FR
2	Northwind	53	X	GRANT	NULL	NULL

Evaluation

- The monitoring script, that can be reused, allows for trouble shooting when users complain the system is slow.
- You can find out which session is blocking other sessions.
- This scenario can occur when for example a GUI application begins a transaction when the “delete customer” dialog is shown. The user indicates which customer to delete. The commit is in the procedure that is called when the ok button is clicked, BUT the user decides to find a coffee before clicking ok.

Exercise

- Scenario: you have a simple inventory system (table PRODUCTS on NORTHWIND) where the first session that starts queries unitsinstock of a single product. The second session wants to update the units in stock.
- The application is built in such a way that when a user queries again the same product in the same transaction the same information is always returned.
- Simulate this scenario in 2 SQL sessions. Create a third session to monitor the blocks that happen.
- What would you have to change if users don't care about getting the same information in the same transaction. Does that influence the blocking? Explain.



4.4. Deadlocks

Deadlocks: The Vicious Cycle

- A deadlock is a special, severe case of blocking.
- It occurs **when two or more sessions (or threads) block each other in a circular fashion**, resulting in a standstill.
- **Resolution:** When a deadlock is detected, **SQL Server automatically terminates (rolls back) one of the sessions**—the Deadlock Victim—allowing the remaining sessions to continue execution.

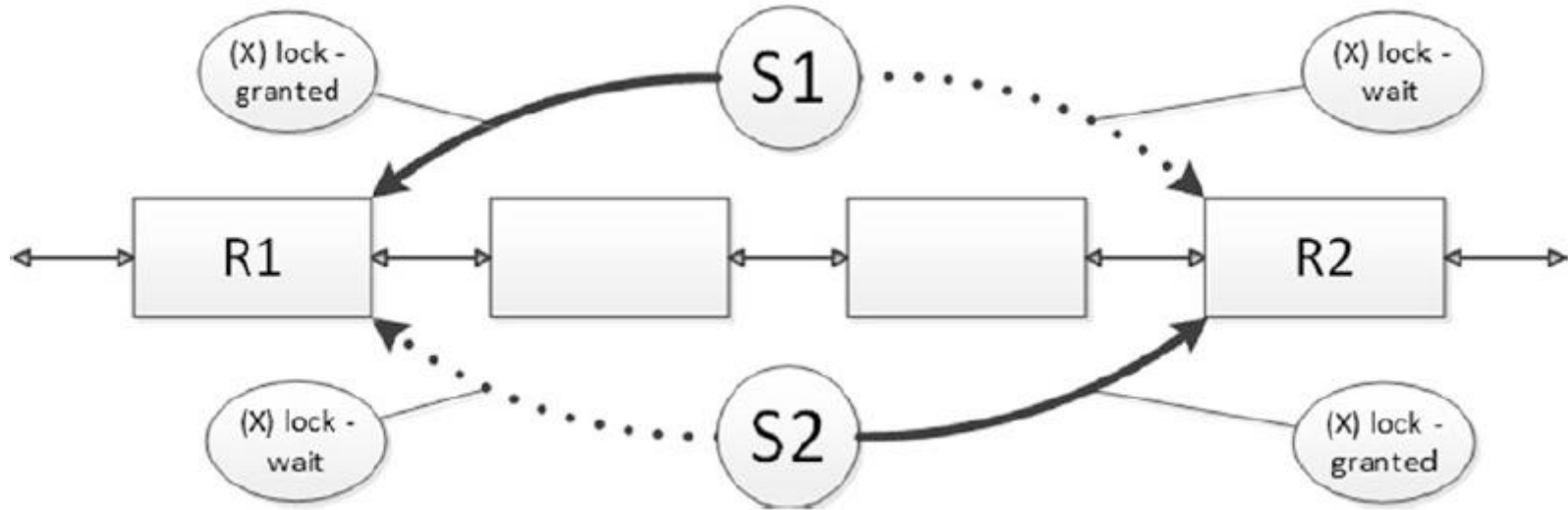
The Classic Deadlock Scenario

- Two or more sessions compete for the same set of resources but attempt to acquire them in a different order.
- Step 1: Initial Contention
 - Session 1: Updates Resource 1 (R1) and acquires an Exclusive (X) lock on R1.
 - Session 2: Updates Resource 2 (R2) and acquires an Exclusive (X) lock on R2.
 - Current State: Both sessions are active, and each holds a lock on a different resource.
- Step 2: The Standoff
 - Session 1: Attempts to read R2, but is blocked because Session 2 already holds an X lock on R2. Session 1 is now waiting for R2.
 - Session 2: Simultaneously attempts to read R1, but is blocked because Session 1 already holds an X lock on R1. Session 2 is now waiting for R1.
- Result: Cycle Deadlock
 - Both sessions are now waiting on each other and can never proceed.
 - This creates a circular wait condition, which is the definition of a classic deadlock.

The Classic Deadlock Scenario

<i>time</i>	T_1	T_2
t_1	begin transaction	
t_2	x-lock(amount _x)	begin transaction
t_3	read(amount _x)	x-lock(amount _y)
t_4	amount _x = amount _x - 50	read(amount _y)
t_5	write(amount _x)	amount _y = amount _y - 30
t_6	x-lock(amount _y)	write(amount _y)
t_7	wait	x-lock(amount _x)
t_8	wait	wait

The Classic Deadlock Scenario



The Deadlock Monitor Task

- **Function:** SQL Server runs an internal system task called the Deadlock Monitor (or Deadlock Detector).
- Check Interval: It routinely **wakes up to check the system for any cycles** (circular waits) indicative of a deadlock.
- Default Interval: The monitor checks for deadlocks approximately **every five seconds**.
- Adaptive Behavior: If deadlocks are actively occurring in the system, this **interval automatically decreases**, allowing the monitor to wake up as often as ten times per second.

Deadlock Resolution

1. **Detection:** When the Deadlock Monitor detects a circular wait, it immediately intervenes.
2. **Termination:** SQL Server chooses one of the involved sessions as the *Deadlock Victim* and terminates (**rolls back**) its transaction.
3. **Error Code:** The victim session receives Error **1205**.
4. **Resolution:** The **rollback releases all locks** held by the victim's transaction, which allows the other surviving sessions to immediately acquire the needed resources and continue their execution.

Choosing the Deadlock Victim

- **Default Policy:** By default, SQL Server **chooses the session that has used the least amount of transaction log space** for its current transaction as the victim.
- **Rationale: Rolling back a smaller transaction is generally less costly** (in terms of processing time and I/O) than rolling back a large one, thus minimizing the overall impact on the system.

Deadlock demo

Run scripts 4.04 Deadlocks - Session 1.sql & 4.04 Deadlocks - Session 2.sql :

```
1  -- Session 1
2
3  use Northwind
4  -- step 1
5  BEGIN TRAN
6  -- Uses default isolation level READ COMMITTED
7  SELECT EmployeeID,Salary from Employees
8  where EmployeeID=1;
9
10 UPDATE Employees SET Salary = Salary+5000
11 where EmployeeID=1;
12
13 SELECT EmployeeID,Salary from Employees
14 where EmployeeID=1;
15 -- simulates delay, allows session 2 to start
16 WAITFOR DELAY '00:00:10'
17
18 -- step 3
19 print 'session 1 continues with step 3'
20 SELECT employeeid,salary from Employees
21 where EmployeeID=2;
22 -- Session 1 has to wait because session 2 holds
23 -- an exclusive lock on this record
24
25 -- step 5
26 COMMIT TRAN -- does not work for victim
27
28 -- End of session 1
```

```
1  -- Session 2
2  -- start session 2 < 10 secs after session 1
3  use Northwind
4  -- step 2
5  BEGIN TRAN
6  -- Uses default isolation level READ COMMITTED
7  SELECT EmployeeID,Salary from Employees
8  where EmployeeID=2;
9
10 UPDATE Employees SET Salary = Salary+5000
11 where EmployeeID=2;
12
13 SELECT EmployeeID,Salary from Employees
14 where EmployeeID=2;
15
16 WAITFOR DELAY '00:00:10'
17
18 -- step 4
19 SELECT employeeid,salary from Employees
20 where EmployeeID=1;
21 -- Session 2 has to wait too because
22 -- session 1 holds an exclusive lock on this record
23 -- A deadlock occurs and one of the sessions
24 -- is chosen as the victim
25
26 -- step 6
27 COMMIT TRAN -- does not work for victim
28
29 -- End of session 2
```

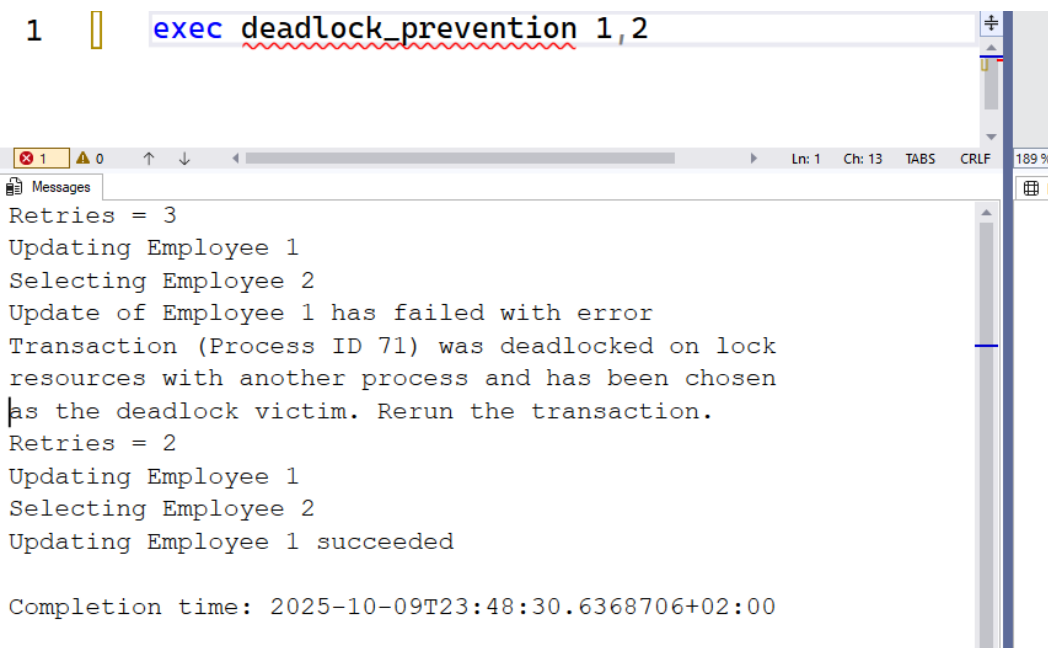
Exercise

1. Write a stored procedure *deadlock_prevention* to run both session 1 and session 2 (so provide 2 parameters).
2. Create 2 sessions and execute in each the stored procedure (with different actual parameters):
 - Session 1: exec deadlock_prevention 1,2
 - Session 2: exec deadlock_prevention 2,1
3. Extend the stored procedure to catch errors. If a deadlock occurs, print a special message. Tip: use
SET @msg = 'my message'
RAISERROR(@msg,0,1) WITH NOWAIT
to display a message iso PRINT because RAISERROR flushes immediately.
4. Further extend the stored procedure to retry the transaction up to 3 times in case of a deadlock. If another error occurs, then quit.

Exercise

You should get an output like this:

```
1  exec deadlock_prevention 1,2
```

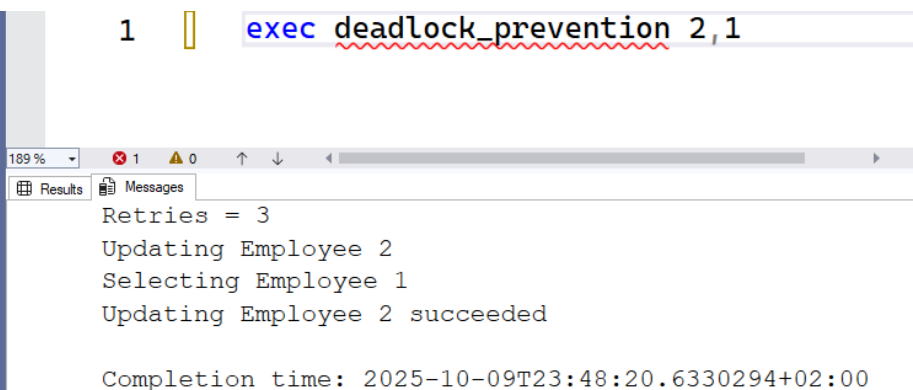


The screenshot shows a SQL Server console window with a command prompt. The command `exec deadlock_prevention 1,2` is entered. The output shows that the first attempt fails due to a deadlock, with a message indicating that the transaction was a victim and should be rerun. The command is then retried, and the output shows that the update of Employee 1 succeeded. The completion time is displayed at the bottom.

```
Retries = 3
Updating Employee 1
Selecting Employee 2
Update of Employee 1 has failed with error
Transaction (Process ID 71) was deadlocked on lock
resources with another process and has been chosen
as the deadlock victim. Rerun the transaction.
Retries = 2
Updating Employee 1
Selecting Employee 2
Updating Employee 1 succeeded

Completion time: 2025-10-09T23:48:30.6368706+02:00
```

```
1  exec deadlock_prevention 2,1
```



The screenshot shows a SQL Server console window with a command prompt. The command `exec deadlock_prevention 2,1` is entered. The output shows that the update of Employee 2 succeeded. The completion time is displayed at the bottom.

```
Retries = 3
Updating Employee 2
Selecting Employee 1
Updating Employee 2 succeeded

Completion time: 2025-10-09T23:48:20.6330294+02:00
```



4.5. Transactions & Triggers

Transactions and triggers

- **In MS SQL Server a trigger is part of the same transaction as the triggering instruction.**
- Inside the trigger this transaction can be ROLLBACKed.
- Although a trigger in SQL Server always occurs after the triggering instruction, that instruction can still be undone in the trigger.
- This mimics the behavior of a before trigger, which exists in other DBMS's.

Transactions and triggers: example

-- If a new record is inserted in OrderDetails => check if the unitPrice is not too low or too high

```
CREATE OR ALTER TRIGGER insertOrderDetails ON OrderDetails FOR insert
AS
```

```
    -- assume only one product is inserted with an INSERT statement
    DECLARE @insertedProductID INT = (SELECT ProductID From inserted)
    DECLARE @insertedUnitPrice Money = (SELECT UnitPrice From inserted)
    DECLARE @unitPriceFromProducts Money =
        (SELECT UnitPrice FROM Products WHERE ProductID = @insertedProductID)
    IF @insertedUnitPrice
        NOT BETWEEN @unitPriceFromProducts * 0.85 AND @unitPriceFromProducts * 1.15
    BEGIN
        ROLLBACK TRANSACTION
        RAISERROR ('The inserted unit price can''t be correct', 14,1)
        RETURN
    END
```

Transactions and triggers: exercise

Scenario: Inventory Price Validation.

- A new business rule requires that whenever a new product is added or an existing product's unit price is updated, the system must enforce two validations before the data is committed to the Products table:
 - Price Floor: The UnitPrice must be greater than \$1.00.
 - Category Check: If the product belongs to the 'Confections' category, its UnitPrice must be at least \$15.00.
- Throw an exception if the validation fails.
- Make sure your code works if >1 records are updated/inserted at once.
- Write test code for all cases.

Sources

Hoofdstukken 2 t.e.m. 5 uit het boek:

Expert SQL Server Transactions and Locking, Dmitri Korotkevitch, Apress, 2018

Code examples:

<https://github.com/Apress/expert-sql-server-transactions-and-locking>

